

IFVG Sniper Model — Pine Script v6

TradingView indicator · 15-minute chart · 703 lines. Copy the code from the raw GitHub link, not from this PDF.

```
1 // @version=6
2 //
3 // IFVG Sniper Model (defaults tuned for the 15-minute chart)
4 // 1. Liquidity sweep — price wicks through a swing / previous-day high-low and closes back inside
5 // 2. V-shape recovery — fast displacement away from the sweep extreme (× ATR)
6 // 3. CISD — close through the open of the delivery series that made the extreme
7 // 4. IFVG — an opposing FVG is closed through by a displacement candle → sniper zone
8 // 5. Retest + sweep — price returns into the IFVG, sweeps minor liquidity and holds → entry
9 // 6. Targets — fixed 1R / 2R (default) with a draw on liquidity required beyond TP1
10 // Filters — higher-timeframe bias, London / New York killzones, max stop size,
11 // discount / premium entry, daily trade & loss limits, one position at a time
12 //
13 indicator("IFVG Sniper Model", "IFVG Sniper", overlay = true, max_bars_back = 500,
14 max_lines_count = 500, max_labels_count = 500, max_boxes_count = 500)
15
16 // ————— Inputs —————
17 signalMode = input.string("Balanced", "Signal mode", options = ["Sniper", "Balanced", "Aggressive",
18 "Custom"],
19 tooltip = "Sniper: strictest, fewest signals.\nBalanced: more signals, keeps the core model (sweep →
20 V → CISD → IFVG → retest).\nAggressive: most signals, filters mostly off.\nCustom: uses every setting
21 below exactly as you set it. In the other modes the starred (*) settings below are set by the mode.")
22 const string G0 = "0 · High-Probability Filters"
23 useHtfIn = input.bool(true, "Trade with higher-timeframe bias *", group = G0,
24 tooltip = "Longs only when the HTF close is above a rising EMA, shorts only below a falling EMA
25 (last closed HTF bar, no repaint).")
26 htfTf = input.timeframe("60", "HTF", group = G0, inline = "htf")
27 htfEmaLen = input.int(50, "EMA", minval = 1, group = G0, inline = "htf")
28 htfSlopeIn = input.bool(true, "HTF EMA must also slope with the trade *", group = G0)
29 useKzIn = input.bool(true, "Entries only inside killzones *", group = G0)
30 kz1In = input.session("0200-0500", "London", group = G0, inline = "kz")
31 kz2In = input.session("0700-1100", "New York", group = G0, inline = "kz")
32 kzTz = input.string("America/New_York", "Killzone timezone", group = G0)
33 dispBodyIn = input.float(0.5, "Min body % of IFVG inversion candle *", minval = 0, maxval = 1, step =
34 0.05, group = G0,
35 tooltip = "The candle that closes through the FVG must be a displacement candle (body / range). 0
36 disables.")
37 pdMinIn = input.float(0.5, "Min retracement into the V leg *", minval = 0, maxval = 1, step = 0.05, group
38 = G0,
39 tooltip = "Entry must sit at least this deep in the range from the sweep extreme to the V high
40 (bull) / low (bear). 0.5 = discount for longs, premium for shorts. 0 disables.")
41 maxDayIn = input.int(2, "Max trades per day *", minval = 1, group = G0, inline = "day")
42 maxLossDayIn = input.int(1, "Stop after N losses *", minval = 1, group = G0, inline = "day")
43 maxSlAtr = input.float(2.0, "Skip entry if stop > N × ATR", minval = 0.2, step = 0.1, group = G0)
44 minSlAtr = input.float(0.3, "Minimum stop size (× ATR)", minval = 0, step = 0.05, group = G0,
45 tooltip = "Stops tighter than this get widened so normal noise does not tag them.")
46
47 const string G1 = "1 · Liquidity Sweep"
48 swingLenIn = input.int(8, "Liquidity swing length *", minval = 2, group = G1,
49 tooltip = "Bars on each side that define a swing high/low (a resting liquidity pool). Higher = only
50 major liquidity.")
51 usePdl = input.bool(true, "Add previous-day high / low as liquidity", group = G1)
52 useAsia = input.bool(true, "Add Asia session high / low as liquidity", group = G1, inline = "asia")
53 asiaSess = input.session("2000-0000", "", group = G1, inline = "asia")
54 maxPools = input.int(30, "Max pools tracked per side", minval = 5, maxval = 100, group = G1)
55 showPools = input.bool(false, "Show resting liquidity levels", group = G1)
56
57 const string G2 = "2 · V-Shape Recovery"
58 vMaxBarsIn = input.int(6, "Max bars from extreme → recovery *", minval = 1, group = G2)
59 vAtrMultIn = input.float(1.5, "Recovery displacement (× ATR) *", minval = 0.1, step = 0.1, group = G2)
60 atrLen = input.int(14, "ATR length", minval = 1, group = G2)
61
62 const string G3 = "3 · CISD + IFVG"
63 alignBarsIn = input.int(12, "Max bars sweep → CISD & IFVG *", minval = 1, group = G3)
64 cisdScan = input.int(15, "CISD candle scan depth", minval = 1, maxval = 100, group = G3)
65 fvgLook = input.int(20, "FVG may form up to N bars before sweep", minval = 1, group = G3)
66 fvgMinAtrIn = input.float(0.1, "Min FVG size (× ATR) *", minval = 0, step = 0.05, group = G3)
67
68 const string G4 = "4 · Retest + Second Sweep"
```

```

60 retestModeIn = input.string("Sweep minor liquidity", "Retest confirmation *", options = ["Sweep minor
liquidity", "Rejection candle", "Touch"], group = G4,
61     tooltip = "Sweep minor liquidity: the retest candle takes the last N bars' low/high and closes back
(strictest).\nRejection candle: the retest candle closes in the trade direction.\nTouch: any retest that
holds the IFVG.")
62 retestMaxIn = input.int(16, "Max bars to wait for retest *", minval = 1, group = G4)
63 minorLen = input.int(3, "Retest sweep lookback (bars)", minval = 1, group = G4,
64     tooltip = "On the retest, price must take the low (bull) / high (bear) of the previous N bars and
close back.")
65
66 const string G5 = "5 · Targets & Management"
67 tpMode = input.string("Fixed R:R", "Target mode", options = ["Fixed R:R", "Liquidity (capped at
R:R)"], group = G5,
68     tooltip = "Fixed R:R: TP1 / TP2 exactly at the R multiples below.\nLiquidity (capped): nearest draw
on liquidity, never further than the R multiples.")
69 rr1 = input.float(1.0, "TP1 (R)", minval = 0.25, step = 0.25, group = G5, inline = "rr")
70 rr2 = input.float(2.0, "TP2 (R)", minval = 0.5, step = 0.25, group = G5, inline = "rr")
71 requireDolIn = input.bool(true, "Require draw on liquidity beyond TP1 *", group = G5,
72     tooltip = "Only enter when an untouched liquidity pool sits at least TP1 away in the trade
direction.")
73 slBuf = input.float(0.1, "SL buffer (× ATR)", minval = 0, step = 0.05, group = G5)
74 partialPct = input.float(50, "Close % at TP1", minval = 0, maxval = 100, step = 5, group = G5)
75 beAfterTp1 = input.bool(true, "Move SL to entry after TP1", group = G5)
76 tradeMax = input.int(96, "Close trade after N bars", minval = 1, group = G5)
77
78 const string G6 = "Display"
79 showBull = input.bool(true, "Bullish setups", group = G6, inline = "d")
80 showBear = input.bool(true, "Bearish setups", group = G6, inline = "d")
81 bullCol = input.color(#089981, "Bull", group = G6, inline = "c")
82 bearCol = input.color(#F23645, "Bear", group = G6, inline = "c")
83 tgtCol = input.color(#26A69A, "Profit", group = G6, inline = "c")
84 slCol = input.color(#EF5350, "Risk", group = G6, inline = "c")
85 showFailed = input.bool(false, "Keep cancelled setups on chart (grey)", group = G6)
86 lblSizeIn = input.string("Normal", "Label size", options = ["Small", "Normal", "Large"], group = G6)
87 showStats = input.bool(true, "Show stats table", group = G6, inline = "t")
88 tablePos = input.string("Top right", "", options = ["Top right", "Bottom right", "Top left", "Bottom
left"], group = G6, inline = "t")
89 string lblSz = lblSizeIn == "Large" ? size.large : lblSizeIn == "Normal" ? size.normal : size.small
90
91 // ----- Effective settings (signal mode) -----
92 bool isSn = signalMode == "Sniper"
93 bool isBa = signalMode == "Balanced"
94 bool isAg = signalMode == "Aggressive"
95 bool isCu = signalMode == "Custom"
96 // Custom Sniper Balanced Aggressive
97 useHtf = isCu ? useHtfIn : isSn ? true : isBa ? true : false
98 htfSlope = isCu ? htfSlopeIn : isSn ? true : false
99 useKz = isCu ? useKzIn : isSn ? true : isBa ? true : false
100 kz1 = isCu ? kz1In : isSn ? "0200-0500" : "0100-0600"
101 kz2 = isCu ? kz2In : isSn ? "0700-1100" : "0700-1300"
102 dispBody = isCu ? dispBodyIn : isSn ? 0.5 : isBa ? 0.35 : 0.0
103 pdMin = isCu ? pdMinIn : isSn ? 0.5 : isBa ? 0.25 : 0.0
104 maxDay = isCu ? maxDayIn : isSn ? 2 : isBa ? 4 : 8
105 maxLossDay = isCu ? maxLossDayIn : isSn ? 1 : isBa ? 2 : 4
106 swingLen = isCu ? swingLenIn : isSn ? 8 : isBa ? 5 : 3
107 vMaxBars = isCu ? vMaxBarsIn : isSn ? 6 : isBa ? 10 : 15
108 vAtrMult = isCu ? vAtrMultIn : isSn ? 1.5 : isBa ? 1.2 : 0.9
109 alignBars = isCu ? alignBarsIn : isSn ? 12 : isBa ? 20 : 30
110 fvgMinAtr = isCu ? fvgMinAtrIn : isSn ? 0.1 : isBa ? 0.05 : 0.0
111 retestMax = isCu ? retestMaxIn : isSn ? 16 : isBa ? 24 : 40
112 requireDol = isCu ? requireDolIn : isSn ? true : isBa ? true : false
113 retestMode = isCu ? retestModeIn : isSn ? "Sweep minor liquidity" : isBa ? "Rejection candle" : "Touch"
114
115 // ----- Core series -----
116 atr = ta.atr(atrLen)
117 minorLow = ta.lowest(low, minorLen)[1]
118 minorHigh = ta.highest(high, minorLen)[1]
119 ph = ta.pivohigh(high, swingLen, swingLen)
120 pl = ta.pivotlow(low, swingLen, swingLen)
121
122 // Higher-timeframe bias from the last CLOSED HTF bar (lookahead + [1] = no repaint)
123 htfData() =>
124     float e = ta.ema(close, htfEmaLen)
125     [close[1], e[1], e[2]]

```

```

126 [htfClose, htfEma, htfEmaPrev] = request.security(syminfo.tickerid, htfTf, htfData(), lookahead =
    barmerge.lookahead_on)
127 bool bullBias = not useHtf or (htfClose > htfEma and (not htfSlope or htfEma > htfEmaPrev))
128 bool bearBias = not useHtf or (htfClose < htfEma and (not htfSlope or htfEma < htfEmaPrev))
129
130 // Killzones
131 bool inKz = not useKz or not na(time(timeframe.period, kz1, kzTz)) or not na(time(timeframe.period, kz2,
    kzTz))
132
133 // Displacement candle (body share of range) for the IFVG inversion
134 float rng = high - low
135 bool bullDisp = close > open and rng > 0 and (close - open) / rng >= dispBody
136 bool bearDisp = close < open and rng > 0 and (open - close) / rng >= dispBody
137
138 // ----- Liquidity pools -----
139 var array<float> bslPx = array.new<float>() // buy-side liquidity (swing highs)
140 var array<int> bslBar = array.new<int>()
141 var array<line> bslLn = array.new<line>()
142 var array<float> sslPx = array.new<float>() // sell-side liquidity (swing lows)
143 var array<int> sslBar = array.new<int>()
144 var array<line> sslLn = array.new<line>()
145
146 addPool(array<float> px, array<int> bx, array<line> ln, float p, int b) =>
147     line l = na
148     if showPools
149         l := line.new(b, p, bar_index, p, color = color.new(color.gray, 40), style = line.style_dotted,
extend = extend.right)
150         px.push(p)
151         bx.push(b)
152         ln.push(l)
153         if px.size() > maxPools
154             px.shift()
155             bx.shift()
156             line.delete(ln.shift())
157
158 // Removes every pool traded through this bar. Returns the most extreme pool that was
159 // swept AND reclaimed (wick through, close back) - d = 1 checks lows, d = -1 checks highs.
160 scanSweeps(array<float> px, array<int> bx, array<line> ln, int d) =>
161     float lvl = na
162     int lb = na
163     int n = px.size()
164     if n > 0
165         for i = n - 1 to 0
166             float p = px.get(i)
167             bool taken = d == 1 ? low < p : high > p
168             if taken
169                 bool reclaimed = d == 1 ? close > p : close < p
170                 if reclaimed and (na(lvl) or d * (lvl - p) > 0)
171                     lvl := p
172                     lb := bx.get(i)
173                     line l = ln.get(i)
174                     if not na(l)
175                         line.set_x2(l, bar_index)
176                         line.set_extend(l, extend.none)
177                     px.remove(i)
178                     bx.remove(i)
179                     ln.remove(i)
180     [lvl, lb]
181
182 // ----- Fair value gaps -----
183 var array<float> bearFvgTop = array.new<float>() // bearish FVGs → become bullish IFVGs
184 var array<float> bearFvgBot = array.new<float>()
185 var array<int> bearFvgBar = array.new<int>()
186 var array<float> bullFvgTop = array.new<float>() // bullish FVGs → become bearish IFVGs
187 var array<float> bullFvgBot = array.new<float>()
188 var array<int> bullFvgBar = array.new<int>()
189
190 addFvg(array<float> top, array<float> bot, array<int> bx, float t, float b, int x) =>
191     top.push(t)
192     bot.push(b)
193     bx.push(x)
194     if top.size() > 100
195         top.shift()
196         bot.shift()

```

```

197         bx.shift()
198
199 // Removes every FVG closed through this bar. Returns the most recent inverted FVG that
200 // formed at/after minBar. d = 1 → bearish FVG closed above, d = -1 → bullish FVG closed below.
201 scanInversions(array<float> top, array<float> bot, array<int> bx, int d, int minBar) =>
202     float zt = na
203     float zb = na
204     int zbar = na
205     int n = top.size()
206     if n > 0
207         for i = n - 1 to 0
208             bool inverted = d == 1 ? close > top.get(i) : close < bot.get(i)
209             if inverted
210                 int fb = bx.get(i)
211                 if fb >= minBar and (na(zbar) or fb > zbar)
212                     zt := top.get(i)
213                     zb := bot.get(i)
214                     zbar := fb
215                 top.remove(i)
216                 bot.remove(i)
217                 bx.remove(i)
218     [zt, zb, zbar]
219
220 // CISD reference: open of the earliest candle in the consecutive opposing series that
221 // delivered price into the current extreme (bull: down-close candles, bear: up-close).
222 cisdLevel(int d) =>
223     float lvl = na
224     int lb = na
225     bool run = false
226     for i = 0 to cisdScan
227         bool opp = d == 1 ? close[i] < open[i] : close[i] > open[i]
228         if opp
229             lvl := open[i]
230             lb := bar_index - i
231             run := true
232         else if run
233             break
234     [lvl, lb]
235
236 // Draw on liquidity: nearest two untouched pools in trade direction, at least minDist away.
237 findTargets(int d, float entry, float minDist) =>
238     array<float> pools = d == 1 ? bs1Px : sslPx
239     float t1 = na
240     float t2 = na
241     for p in pools
242         if d * (p - entry) >= minDist
243             if na(t1) or d * (t1 - p) > 0
244                 t2 := t1
245                 t1 := p
246             else if p != t1 and (na(t2) or d * (t2 - p) > 0)
247                 t2 := p
248     [t1, t2]
249
250 // ----- Setup state machine -----
251 // st: 0 idle · 1 swept (waiting V + CISD + IFVG) · 2 IFVG armed (waiting retest) · 3 in trade
252 type Setup
253     int dir
254     int st = 0
255     float lvl
256     int lvlBar
257     int sweepBar
258     float ext
259     int extBar
260     bool vDone = false
261     int vBar
262     float vPx
263     float cisd
264     int cisdBar
265     bool cisdDone = false
266     int cisdConfBar
267     bool zDone = false
268     float zTop
269     float zBot
270     int zBar

```

```

271     int    armBar
272     float far        // V-leg extreme in trade direction (for discount / premium)
273     float entry
274     float sl
275     float tp1
276     float tp2
277     float risk
278     float r1
279     float r2
280     bool  tp1Hit    = false
281     int   entryBar
282     box   zBox
283     line  dSweepLn
284     label dSweepLb
285     label dVLb
286     line  dCisdLn
287     label dCisdLb
288     box   bxRisk
289     box   bxRew
290     line  lT1
291     label bS
292     label bT1
293     label bT2
294
295 method reset(Setup s) =>
296     s.st      := 0
297     s.vDone   := false
298     s.cisdDone := false
299     s.zDone   := false
300     s.tp1Hit  := false
301     s.zBox    := na
302     s.dSweepLn := na
303     s.dSweepLb := na
304     s.dVLb    := na
305     s.dCisdLn := na
306     s.dCisdLb := na
307     s.bxRisk  := na
308     s.bxRew   := na
309     s.lT1     := na
310     s.bS      := na
311     s.bT1     := na
312     s.bT2     := na
313     true
314
315 method start(Setup s, float lvl, int lvlBar) =>
316     float adv = s.dir == 1 ? low : high
317     if s.st == 0
318         [c, cb] = cisdLevel(s.dir)
319         s.st     := 1
320         s.lvl    := lvl
321         s.lvlBar := lvlBar
322         s.sweepBar := bar_index
323         s.ext     := adv
324         s.extBar  := bar_index
325         s.cisd    := c
326         s.cisdBar := cb
327         s.far     := s.dir == 1 ? high : low
328     else if s.st == 1 and s.dir * (s.lvl - lvl) > 0
329         s.lvl    := lvl
330         s.lvlBar := lvlBar
331     true
332
333 // "+1.5R" / "-1R"
334 fmtR(float r) => (r >= 0 ? "+" : "") + str.toString(r, "#.##") + "R"
335
336 method setTradeX(Setup s, int x) =>
337     box.set_right(s.bxRisk, x)
338     box.set_right(s.bxRew, x)
339     line.set_x2(s.lT1, x)
340     label.set_x(s.bS, x)
341     label.set_x(s.bT1, x)
342     label.set_x(s.bT2, x)
343     true
344

```

```

345 // bias: HTF bias agrees with this direction · allow: killzone, daily limits and no open opposite trade
346 method update(Setup s, float atrV, float minorLvl, float zt, float zb, int zbar, bool bias, bool allow)
=>
347     int    d    = s.dir
348     color c    = d == 1 ? bullCol : bearCol
349     float adv = d == 1 ? low : high
350     float fav = d == 1 ? high : low
351     bool evArm = false
352     bool evEntry = false
353     bool evTp1 = false
354     bool evTp2 = false
355     bool evSl = false
356     bool evBe = false
357     float resR = na // realised R when the trade closes on this bar
358     float part = partialPct / 100
359
360     if s.st == 1
361         // New extreme → any partial confirmation is void, rebuild from the new low/high
362         if d * (s.ext - adv) > 0
363             [cl, cb] = cisdLevel(d)
364             s.ext := adv
365             s.extBar := bar_index
366             s.vDone := false
367             s.cisdDone := false
368             s.zDone := false
369             s.cisd := cl
370             s.cisdBar := cb
371             s.far := fav
372             s.far := na(s.far) or d * (fav - s.far) > 0 ? fav : s.far
373             if not na(zbar)
374                 s.zTop := zt
375                 s.zBot := zb
376                 s.zBar := zbar
377                 s.zDone := true
378             if not s.vDone and bar_index - s.extBar <= vMaxBars and d * (fav - s.ext) >= vAtrMult * atrV and
d * (close - s.lvl) > 0
379                 s.vDone := true
380                 s.vBar := bar_index
381                 s.vPx := fav
382             if not s.cisdDone and not na(s.cisd) and d * (close - s.cisd) > 0
383                 s.cisdDone := true
384                 s.cisdConfBar := bar_index
385
386             if s.vDone and s.cisdDone and s.zDone
387                 s.st := 2
388                 s.armBar := bar_index
389                 evArm := true
390                 color cf = color.new(c, 20)
391                 s.dSweepLn := line.new(s.lvlBar, s.lvl, s.sweepBar, s.lvl, color = c, style =
line.style_dashed)
392                 s.dSweepLb := label.new(s.extBar, s.ext, "1 Sweep", color = cf, textcolor = color.white,
393                     style = d == 1 ? label.style_label_up : label.style_label_down, size = lblSz,
394                     tooltip = (d == 1 ? "Sell-side" : "Buy-side") + " liquidity at " + str.tostring(s.lvl,
format.mintick) + " taken and reclaimed")
395                 s.dVLb := label.new(s.vBar, s.vPx, "2 V", color = cf, textcolor = color.white,
396                     style = d == 1 ? label.style_label_down : label.style_label_up, size = lblSz,
397                     tooltip = "V-shape recovery: fast displacement away from the sweep")
398                 s.dCisdLn := line.new(s.cisdBar, s.cisd, s.cisdConfBar, s.cisd, color = c, width = 2)
399                 s.dCisdLb := label.new(s.cisdBar, s.cisd, "3 CISD", color = cf, textcolor = color.white,
400                     style = label.style_label_right, size = lblSz,
401                     tooltip = "Change in state of delivery: close through " + str.tostring(s.cisd,
format.mintick))
402                 s.zBox := box.new(s.zBar, math.max(s.zTop, s.zBot), bar_index, math.min(s.zTop, s.zBot),
403                     border_color = c, bgcolor = color.new(c, 82), text = "4 IFVG", text_color = c,
404                     text_size = lblSz, text_halign = text.align_left, text_valign = text.align_center)
405             else if bar_index - s.sweepBar > alignBars or (not s.vDone and bar_index - s.extBar > vMaxBars)
406                 s.reset()
407
408     else if s.st == 2
409         float zNear = d == 1 ? s.zTop : s.zBot
410         float zFar = d == 1 ? s.zBot : s.zTop
411         box.set_right(s.zBox, bar_index)
412         s.far := d * (fav - s.far) > 0 ? fav : s.far
413         if bar_index > s.armBar

```

```

414     bool broke    = d * (close - zFar) < 0 or d * (adv - s.ext) < 0
415     bool expired  = bar_index - s.armBar > retestMax
416     bool touched  = d * (adv - zNear) <= 0
417     bool sweep2   = d * (adv - minorLvl) < 0 and d * (close - minorLvl) > 0
418     bool confirm  = retestMode == "Sweep minor liquidity" ? sweep2 : retestMode == "Rejection
candle" ? d * (close - open) > 0 : true
419     bool holds    = d * (close - zFar) >= 0
420     if broke or expired
421         if showFailed
422             box.set_bgcolor(s.zBox, color.new(color.gray, 88))
423             box.set_border_color(s.zBox, color.gray)
424             box.set_text_color(s.zBox, color.gray)
425             box.set_text(s.zBox, "IFVG cancelled")
426         else
427             box.delete(s.zBox)
428             line.delete(s.dSweepLn)
429             label.delete(s.dSweepLb)
430             label.delete(s.dVLb)
431             line.delete(s.dCisdLn)
432             label.delete(s.dCisdLb)
433         s.reset()
434     else if touched and confirm and holds and bias and allow
435         float entry = close
436         float legSz = math.abs(s.far - s.ext)
437         float depth = legSz > 0 ? d * (s.far - entry) / legSz : 0.0
438         float risk  = math.max(math.abs(entry - (adv - d * slBuf * atrV)), minSlAtr * atrV)
439         [dol, dol2] = findTargets(d, entry, rr1 * risk)
440         bool okSize = risk <= maxSlAtr * atrV
441         bool okDol  = not requireDol or not na(dol)
442         bool okPd   = depth >= pdMin
443         // A retest that fails the stop-size / DOL / discount filters is skipped; the zone stays
armed.
444         if okSize and okDol and okPd
445             s.st      := 3
446             s.entryBar := bar_index
447             s.entry    := entry
448             s.risk     := risk
449             s.sl       := entry - d * risk
450             float cap1 = entry + d * rr1 * risk
451             float cap2 = entry + d * rr2 * risk
452             if tpMode == "Fixed R:R"
453                 s.tp1 := cap1
454                 s.tp2 := cap2
455             else
456                 [t1, t2] = findTargets(d, entry, 0.5 * risk)
457                 s.tp1 := na(t1) or d * (t1 - cap1) > 0 ? cap1 : t1
458                 s.tp2 := na(t2) or d * (t2 - cap2) > 0 ? cap2 : t2
459                 if d * (s.tp2 - s.tp1) <= 0
460                     s.tp2 := entry + d * math.max(rr2 * risk, math.abs(s.tp1 - entry) + 0.5 *
risk)
461             s.r1 := math.abs(s.tp1 - entry) / risk
462             s.r2 := math.abs(s.tp2 - entry) / risk
463             evEntry := true
464
465             int x2 = bar_index + 5
466             string tip = "Entry " + str.tostring(s.entry, format.mintick) +
467                 "\nSL " + str.tostring(s.sl, format.mintick) +
468                 "\nTP1 " + str.tostring(s.tp1, format.mintick) + " (" + str.tostring(s.r1,
469 "#.##") + "R)" +
470                 "\nTP2 " + str.tostring(s.tp2, format.mintick) + " (" + str.tostring(s.r2,
471 "#.##") + "R)" +
472                 "\nDraw on liquidity " + (na(dol) ? "none beyond TP1" : str.tostring(dol,
format.mintick)) +
473                 "\nRetracement " + str.tostring(depth * 100, "#") + "% of V leg"
474             label.new(bar_index, adv, d == 1 ? "5 BUY" : "5 SELL", color = c, textcolor =
color.white,
475                 style = d == 1 ? label.style_label_up : label.style_label_down, size = lblSz,
tooltip = tip)
476             box.set_border_width(s.zBox, 2)
477             s.bxRisk := box.new(bar_index, math.max(s.entry, s.sl), x2, math.min(s.entry, s.sl),
border_color = color.new(slCol, 50), bgcolor = color.new(slCol, 80))
478             s.bxRew := box.new(bar_index, math.max(s.entry, s.tp2), x2, math.min(s.entry,
s.tp2),
border_color = color.new(tgtCol, 50), bgcolor = color.new(tgtCol, 80))

```

```

479         s.lT1 := line.new(bar_index, s.tp1, x2, s.tp1, color = tgtCol, style =
line.style_dashed)
480         s.bS := label.new(x2, s.sl, "SL " + str.tostring(s.sl, format.mintick), color =
color(na),
481             textcolor = slCol, style = label.style_label_left, size = lblSz)
482         s.bT1 := label.new(x2, s.tp1, "TP1 " + str.tostring(s.r1, "#.#") + "R", color =
color(na),
483             textcolor = tgtCol, style = label.style_label_left, size = lblSz)
484         s.bT2 := label.new(x2, s.tp2, "TP2 " + str.tostring(s.r2, "#.#") + "R", color =
color(na),
485             textcolor = tgtCol, style = label.style_label_left, size = lblSz)
486
487     else if s.st == 3
488         if bar_index > s.entryBar
489             s.setTradeX(bar_index + 5)
490             bool hitSl = d * (adv - s.sl) <= 0
491             bool hitTp1 = d * (fav - s.tp1) >= 0
492             bool hitTp2 = d * (fav - s.tp2) >= 0
493             // Conservative: if SL and a target print on the same bar, the stop is assumed first
494             if hitSl
495                 if s.tp1Hit
496                     evBe := true
497                     resR := part * s.r1 + (1 - part) * d * (s.sl - s.entry) / s.risk
498                 else
499                     evSl := true
500                     resR := -1.0
501                 label.new(bar_index, adv, (s.tp1Hit ? "BE " : "x SL ") + fmtR(resR), color = s.tp1Hit ?
color.gray : slCol,
502                     textcolor = color.white, style = d == 1 ? label.style_label_up :
label.style_label_down, size = lblSz)
503                 s.setTradeX(bar_index)
504                 s.reset()
505             else if hitTp2
506                 if not s.tp1Hit
507                     evTp1 := true
508                     evTp2 := true
509                     resR := part * s.r1 + (1 - part) * s.r2
510                 label.new(bar_index, s.tp2, "✓ TP2 " + fmtR(resR), color = tgtCol, textcolor =
color.white,
511                     style = d == 1 ? label.style_label_down : label.style_label_up, size = lblSz)
512                 s.setTradeX(bar_index)
513                 s.reset()
514             else
515                 if hitTp1 and not s.tp1Hit
516                     s.tp1Hit := true
517                     evTp1 := true
518                 label.new(bar_index, s.tp1, "✓ TP1", color = color.new(tgtCol, 30), textcolor =
color.white,
519                     style = d == 1 ? label.style_label_down : label.style_label_up, size = lblSz)
520                 if beAfterTp1
521                     s.sl := s.entry
522                     box.set_bgcolor(s.bxRisk, color.new(color.gray, 90))
523                     box.set_border_color(s.bxRisk, color.new(color.gray, 70))
524                     label.set_y(s.bS, s.entry)
525                     label.set_text(s.bS, "SL moved to entry")
526                 if bar_index - s.entryBar >= tradeMax
527                     float openR = d * (close - s.entry) / s.risk
528                     resR := s.tp1Hit ? part * s.r1 + (1 - part) * openR : openR
529                     label.new(bar_index, close, "Time exit " + fmtR(resR), color = color.gray, textcolor
= color.white,
530                         style = label.style_label_left, size = lblSz)
531                     s.setTradeX(bar_index)
532                     s.reset()
533
534         [evArm, evEntry, evTp1, evTp2, evSl, evBe, resR]
535
536 // ----- Engine -----
537 var Setup bull = Setup.new(1)
538 var Setup bear = Setup.new(-1)
539
540 // 1 · register new liquidity pools
541 if not na(ph)
542     addPool(bslPx, bslBar, bslLn, ph, bar_index - swingLen)
543 if not na(pl)

```

```

544     addPool(sslPx, sslBar, sslLn, pl, bar_index - swingLen)
545
546 // previous-day high / low become liquidity pools at the start of each new day
547 var float dH = na
548 var float dL = na
549 var int   dHB = na
550 var int   dLB = na
551 if timeframe.change("D")
552     if usePdl and not na(dH)
553         addPool(bslPx, bslBar, bslLn, dH, dHB)
554         addPool(sslPx, sslBar, sslLn, dL, dLB)
555     dH := high
556     dHB := bar_index
557     dL := low
558     dLB := bar_index
559 else
560     if na(dH) or high > dH
561         dH := high
562         dHB := bar_index
563     if na(dL) or low < dL
564         dL := low
565         dLB := bar_index
566
567 // Asia session high / low become liquidity pools when the session ends
568 bool inAsia = not na(time(timeframe.period, asiaSess, kzTz))
569 var float aH = na
570 var float aL = na
571 var int   aHB = na
572 var int   aLB = na
573 if inAsia
574     if not inAsia[1] or na(aH)
575         aH := high
576         aHB := bar_index
577         aL := low
578         aLB := bar_index
579     else
580         if high > aH
581             aH := high
582             aHB := bar_index
583         if low < aL
584             aL := low
585             aLB := bar_index
586 else if inAsia[1] and useAsia and not na(aH)
587     addPool(bslPx, bslBar, bslLn, aH, aHB)
588     addPool(sslPx, sslBar, sslLn, aL, aLB)
589
590 // daily trade / loss limits (reset with the trading day)
591 var int dayTrades = 0
592 var int dayLosses = 0
593 if timeframe.change("D")
594     dayTrades := 0
595     dayLosses := 0
596 bool dayOk = dayTrades < maxDay and dayLosses < maxLossDay
597
598 // 2 · liquidity sweeps start (or deepen) a setup
599 [sslLv1, sslB] = scanSweeps(sslPx, sslBar, sslLn, 1)
600 [bslLv1, bslB] = scanSweeps(bslPx, bslBar, bslLn, -1)
601 if showBull and bullBias and not na(sslLv1)
602     bull.start(sslLv1, sslB)
603 if showBear and bearBias and not na(bslLv1)
604     bear.start(bslLv1, bslB)
605
606 // 3 · FVG inversions (only FVGs formed around the sweep, inverted by a displacement candle)
607 int bullMin = bull.st == 1 and bullDisp ? bull.sweepBar - fvgLook : bar_index + 1
608 int bearMin = bear.st == 1 and bearDisp ? bear.sweepBar - fvgLook : bar_index + 1
609 [bzt, bzb, bzbar] = scanInversions(bearFvgTop, bearFvgBot, bearFvgBar, 1, bullMin)
610 [szt, szb, szbar] = scanInversions(bullFvgTop, bullFvgBot, bullFvgBar, -1, bearMin)
611
612 if high < low[2] and low[2] - high >= fvgMinAtr * atr
613     addFvg(bearFvgTop, bearFvgBot, bearFvgBar, low[2], high, bar_index - 1)
614 if low > high[2] and low - high[2] >= fvgMinAtr * atr
615     addFvg(bullFvgTop, bullFvgBot, bullFvgBar, low, high[2], bar_index - 1)
616
617 // 4 · advance state machines

```

```

618 // one position at a time: a direction may not enter while the other is in a trade
619 [bArm, bEntry, bTp1, bTp2, bSl, bBe, bR] = bull.update(atr, minorLow, bzt, bzb, bzbar, bullBias, inKz and
day0k and bear.st != 3)
620 [sArm, sEntry, sTp1, sTp2, sSl, sBe, sR] = bear.update(atr, minorHigh, szt, szb, szbar, bearBias, inKz
and day0k and bull.st != 3)
621 dayTrades += (bEntry ? 1 : 0) + (sEntry ? 1 : 0)
622 dayLosses += (not na(bR) and bR < 0 ? 1 : 0) + (not na(sR) and sR < 0 ? 1 : 0)
623
624 // ----- Stats -----
625 var int nTrades = 0
626 var int nClosed = 0
627 var int nWin = 0
628 var int nTp1 = 0
629 var int nTp2 = 0
630 var int nLoss = 0
631 var float netR = 0.0
632 nTrades += (bEntry ? 1 : 0) + (sEntry ? 1 : 0)
633 nTp1 += (bTp1 ? 1 : 0) + (sTp1 ? 1 : 0)
634 nTp2 += (bTp2 ? 1 : 0) + (sTp2 ? 1 : 0)
635 nLoss += (bSl ? 1 : 0) + (sSl ? 1 : 0)
636 for r in array.from(bR, sR)
637     if not na(r)
638         nClosed += 1
639         nWin += r > 0 ? 1 : 0
640         netR += r
641
642 statRow(table t, int row, string k, string v, color c) =>
643     table.cell(t, 0, row, k, text_color = color.silver, text_size = size.small, text_halign =
text.align_left)
644     table.cell(t, 1, row, v, text_color = c, text_size = size.small, text_halign = text.align_right)
645
646 pct(int a, int b) => b > 0 ? str.toString(100.0 * a / b, "#") + "%" : "-"
647
648 stateName(Setup s) =>
649     s.st == 0 ? "Idle" : s.st == 1 ? "Swept – waiting V/CISD/IFVG" : s.st == 2 ? "IFVG armed – waiting
retest" : "In trade"
650
651 string tPos = tablePos == "Bottom right" ? position.bottom_right : tablePos == "Top left" ?
position.top_left :
652     tablePos == "Bottom left" ? position.bottom_left : position.top_right
653 var table stats = table.new(tPos, 2, 13, bgcolor = color.new(#131722, 10), border_width = 1)
654 if showStats and barstate.islast
655     table.cell(stats, 0, 0, "IFVG Sniper · " + signalMode + " · " + timeframe.period + (timeframe.period
== "15" ? "" : " (tuned for 15)"),
656         text_color = color.white, text_size = size.small)
657     table.merge_cells(stats, 0, 0, 1, 0)
658     statRow(stats, 1, "Trades closed", str.toString(nClosed) + " / " + str.toString(nTrades),
color.white)
659     statRow(stats, 2, "Win rate (R > 0)", pct(nWin, nClosed), tgtCol)
660     statRow(stats, 3, "TP1 reached", pct(nTp1, nClosed), tgtCol)
661     statRow(stats, 4, "TP2 reached", pct(nTp2, nClosed), tgtCol)
662     statRow(stats, 5, "Stopped before TP1", pct(nLoss, nClosed), slCol)
663     statRow(stats, 6, "Net R", str.toString(netR, "#.##"), netR >= 0 ? tgtCol : slCol)
664     statRow(stats, 7, "Avg R / trade", nClosed > 0 ? str.toString(netR / nClosed, "#.##") : "-", netR >=
0 ? tgtCol : slCol)
665     statRow(stats, 8, "HTF bias", not useHtf ? "off" : bullBias ? "Bullish" : bearBias ? "Bearish" :
"Neutral – no trades",
666         not useHtf ? color.gray : bullBias ? bullCol : bearBias ? bearCol : color.gray)
667     statRow(stats, 9, "Killzone", not useKz ? "off" : inKz ? "Open" : "Closed", inKz ? tgtCol :
color.gray)
668     statRow(stats, 10, "Today", str.toString(dayTrades) + " trades · " + str.toString(dayLosses) + "
losses" + (day0k ? "" : " – done"),
669         day0k ? color.white : slCol)
670     statRow(stats, 11, "Bull", stateName(bull), bullCol)
671     statRow(stats, 12, "Bear", stateName(bear), bearCol)
672
673 // ----- Data window -----
674 plot(bull.st == 3 ? bull.entry : na, "Bull entry", bullCol, display = display.data_window)
675 plot(bull.st == 3 ? bull.sl : na, "Bull SL", slCol, display = display.data_window)
676 plot(bull.st == 3 ? bull.tp1 : na, "Bull TP1", tgtCol, display = display.data_window)
677 plot(bull.st == 3 ? bull.tp2 : na, "Bull TP2", tgtCol, display = display.data_window)
678 plot(bear.st == 3 ? bear.entry : na, "Bear entry", bearCol, display = display.data_window)
679 plot(bear.st == 3 ? bear.sl : na, "Bear SL", slCol, display = display.data_window)
680 plot(bear.st == 3 ? bear.tp1 : na, "Bear TP1", tgtCol, display = display.data_window)

```

```

681 plot(bear.st == 3 ? bear.tp2 : na, "Bear TP2", tgtCol, display = display.data_window)
682
683 // ----- Alerts -----
684 alertcondition(bArm, "Bull IFVG armed", "IFVG Sniper: bullish sweep + V + CISD + IFVG armed on
{{ticker}} ({{interval}})")
685 alertcondition(bEntry, "Bull entry", "IFVG Sniper: BUY on {{ticker}} @ {{close}} - IFVG retest +
sweep")
686 alertcondition(bTp1, "Bull TP1", "IFVG Sniper: bullish TP1 hit on {{ticker}}")
687 alertcondition(bTp2, "Bull TP2", "IFVG Sniper: bullish TP2 hit on {{ticker}}")
688 alertcondition(bSl, "Bull stop", "IFVG Sniper: bullish stop hit on {{ticker}}")
689 alertcondition(sArm, "Bear IFVG armed", "IFVG Sniper: bearish sweep + V + CISD + IFVG armed on
{{ticker}} ({{interval}})")
690 alertcondition(sEntry, "Bear entry", "IFVG Sniper: SELL on {{ticker}} @ {{close}} - IFVG retest +
sweep")
691 alertcondition(sTp1, "Bear TP1", "IFVG Sniper: bearish TP1 hit on {{ticker}}")
692 alertcondition(sTp2, "Bear TP2", "IFVG Sniper: bearish TP2 hit on {{ticker}}")
693 alertcondition(sSl, "Bear stop", "IFVG Sniper: bearish stop hit on {{ticker}}")
694
695 if bEntry
696     alert("IFVG Sniper BUY " + syminfo.ticker + " | entry " + str.tostring(bull.entry, format.mintick) +
697         " | SL " + str.tostring(bull.sl, format.mintick) + " | TP1 " + str.tostring(bull.tp1,
format.mintick) +
698         " | TP2 " + str.tostring(bull.tp2, format.mintick), alert.freq_once_per_bar_close)
699 if sEntry
700     alert("IFVG Sniper SELL " + syminfo.ticker + " | entry " + str.tostring(bear.entry, format.mintick) +
701         " | SL " + str.tostring(bear.sl, format.mintick) + " | TP1 " + str.tostring(bear.tp1,
format.mintick) +
702         " | TP2 " + str.tostring(bear.tp2, format.mintick), alert.freq_once_per_bar_close)
703

```